In [1]:
```
pip install pandas scikit-learn nltk
```

```
Requirement already satisfied: pandas in ./anaconda3/lib/python3.9/site-packages (2.0.
3)
Requirement already satisfied: scikit-learn in ./anaconda3/lib/python3.9/site-packages
(1.3.0)
Requirement already satisfied: nltk in ./anaconda3/lib/python3.9/site-packages (3.8.1)
Requirement already satisfied: python-dateutil>=2.8.2 in ./anaconda3/lib/python3.9/sit
e-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in ./anaconda3/lib/python3.9/site-packages
(from pandas) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in ./anaconda3/lib/python3.9/site-packag
es (from pandas) (2023.3)
Requirement already satisfied: numpy>=1.20.3 in ./anaconda3/lib/python3.9/site-package
s (from pandas) (1.23.5)
Requirement already satisfied: scipy>=1.5.0 in ./anaconda3/lib/python3.9/site-packages
(from scikit-learn) (1.11.4)
Requirement already satisfied: joblib>=1.1.1 in ./anaconda3/lib/python3.9/site-package
s (from scikit-learn) (1.2.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in ./anaconda3/lib/python3.9/site-
packages (from scikit-learn) (2.2.0)
Requirement already satisfied: click in ./anaconda3/lib/python3.9/site-packages (from
nltk) (8.1.7)
Requirement already satisfied: regex>=2021.8.3 in ./anaconda3/lib/python3.9/site-packa
ges (from nltk) (2023.10.3)
Requirement already satisfied: tqdm in ./anaconda3/lib/python3.9/site-packages (from n
ltk) (4.65.0)
Requirement already satisfied: six>=1.5 in ./anaconda3/lib/python3.9/site-packages (fr
om python-dateutil>=2.8.2->pandas) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

In [15]:
```
import pandas as pd

data = pd.read_csv("Reviews.csv")
data.head()
```

Out[15]:

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	Helpfulness
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	

In [17]:
```
data.columns
```

Out[17]:
```
Index(['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator',
       'HelpfulnessDenominator', 'Score', 'Time', 'Summary', 'Text'],
      dtype='object')
```

In [19]:
```
data = data[['Text', 'Score']]  # adjust column names if needed

data = data[data['Score'] != 3]
data['sentiment'] = data['Score'].apply(lambda x: 1 if x >= 4 else 0)
```

In [21]:
```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

data['clean_text'] = data['Text'].apply(clean_text)
```

In [23]:
```
from sklearn.model_selection import train_test_split

X = data['clean_text']
y = data['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [25]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    stop_words='english'
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

In [26]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)
```

Out[26]:
```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

In [27]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

y_pred = model.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

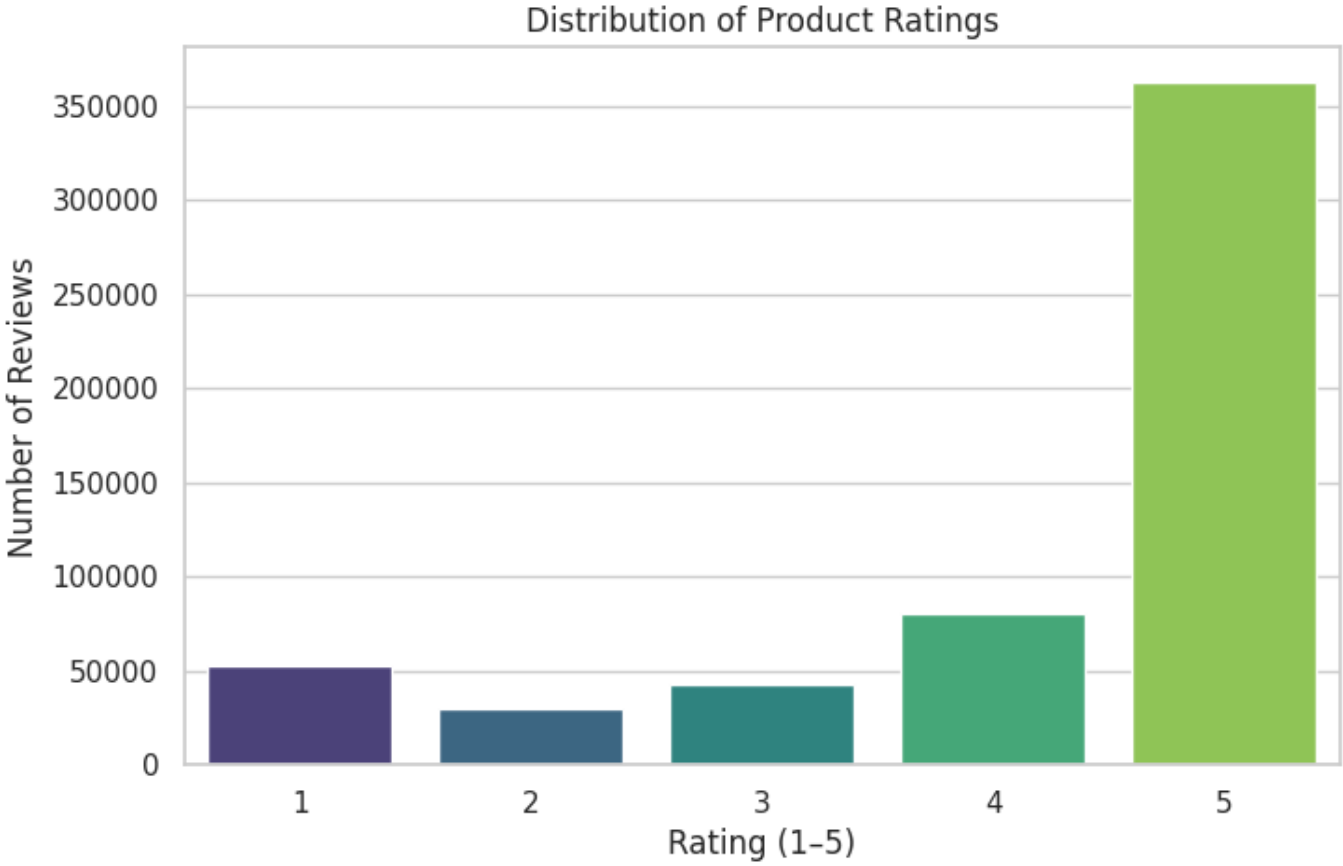
```
Accuracy: 0.9304983691982922
              precision    recall  f1-score   support

           0       0.84      0.68      0.75     16407
           1       0.94      0.98      0.96     88756

    accuracy                           0.93    105163
   macro avg       0.89      0.83      0.86    105163
weighted avg       0.93      0.93      0.93    105163
```

In [28]:
```
def predict_sentiment(review):
    review = clean_text(review)
    vec = vectorizer.transform([review])
    prediction = model.predict(vec)[0]
    return "Positive" if prediction == 1 else "Negative"

predict_sentiment("This product is amazing and works perfectly!")
```

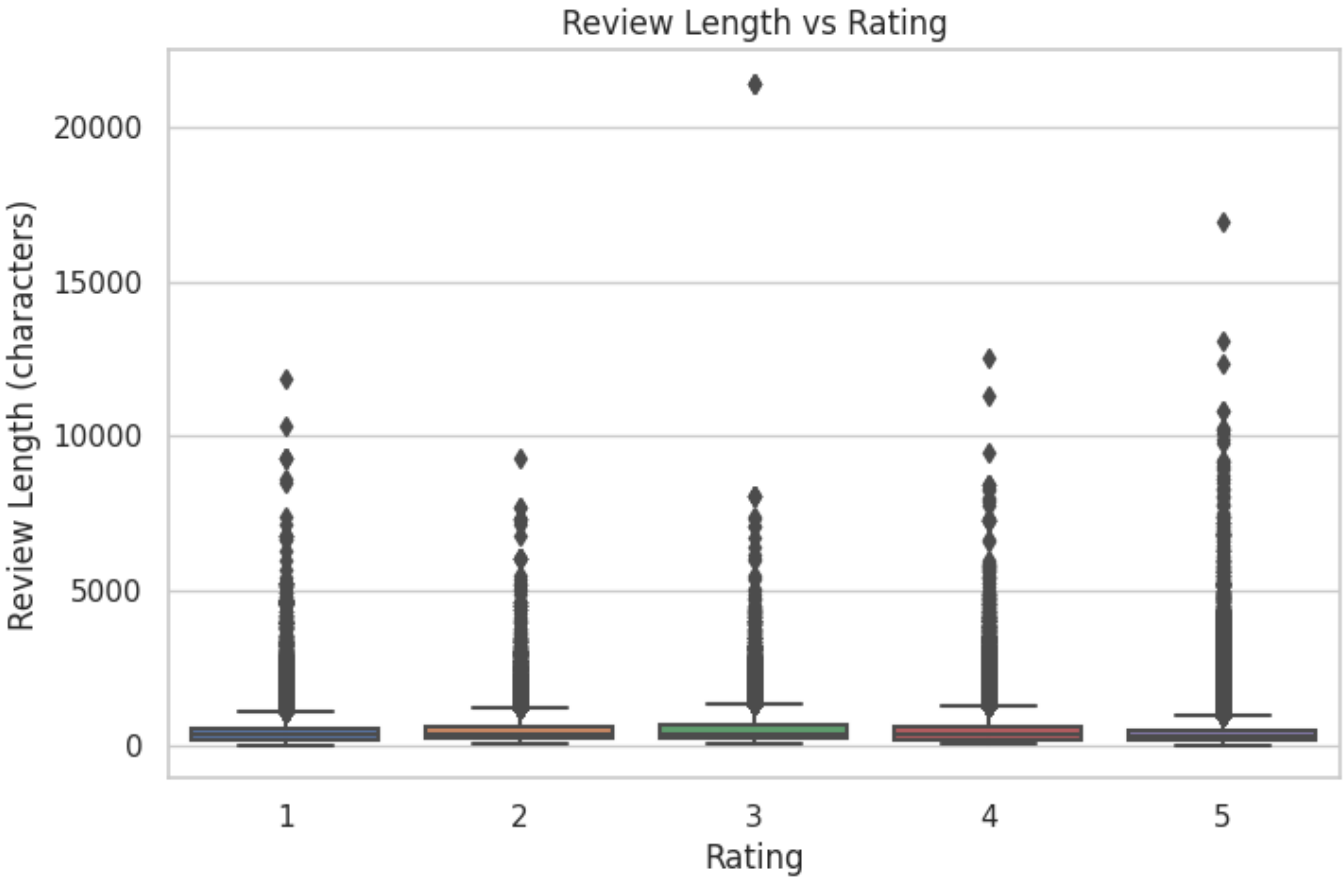
Out[28]:
```
'Positive'
```

In [33]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
data = pd.read_csv("Reviews.csv")
```

In [35]:
```
plt.figure(figsize=(8,5))
sns.countplot(x='Score', data=data, palette='viridis')
plt.title("Distribution of Product Ratings")
plt.xlabel("Rating (1–5)")
plt.ylabel("Number of Reviews")
plt.show()
```

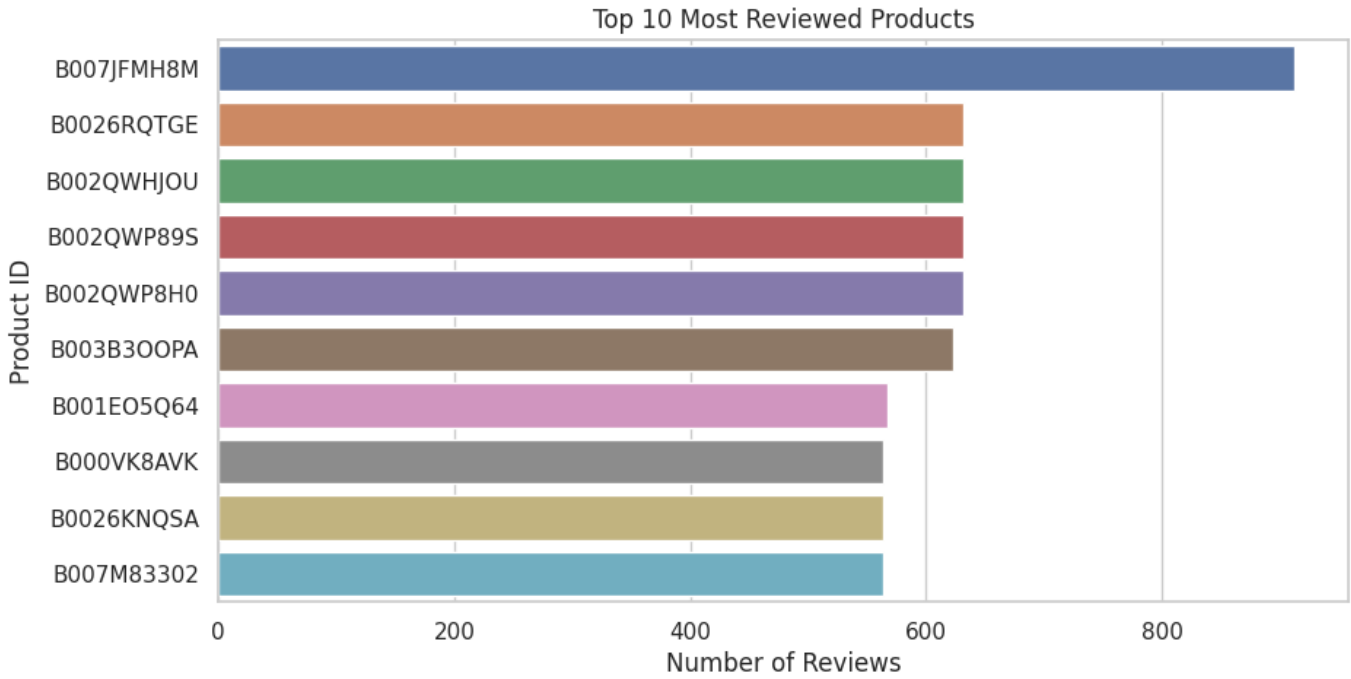


In [37]:
```
data['Review_Length'] = data['Text'].astype(str).apply(len)

plt.figure(figsize=(8,5))
sns.boxplot(x='Score', y='Review_Length', data=data)
plt.title("Review Length vs Rating")
plt.xlabel("Rating")
plt.ylabel("Review Length (characters)")
plt.show()
```

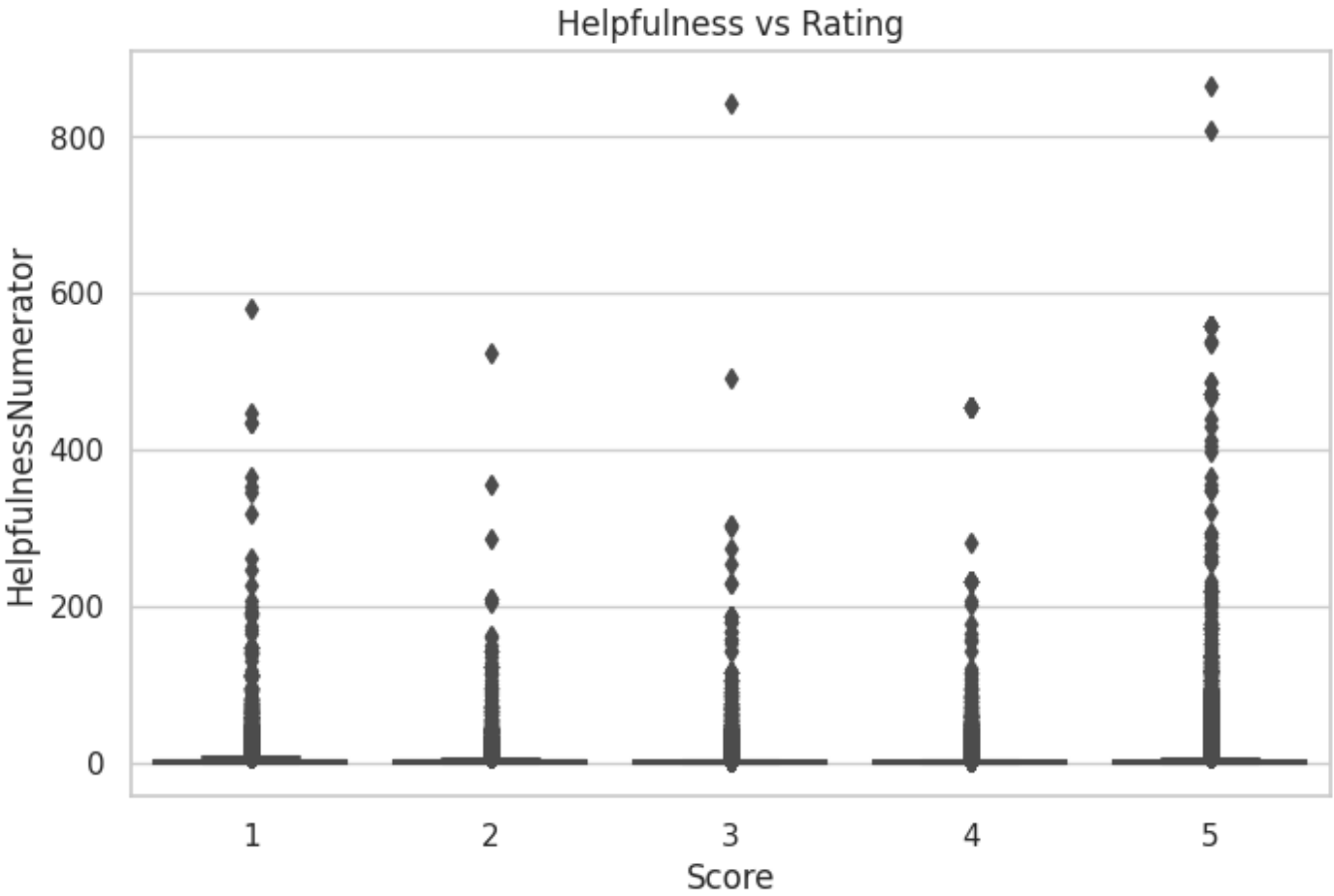


In [46]:
```
top_products = data['ProductId'].value_counts().head(10)
```

```
plt.figure(figsize=(10,5))
sns.barplot(x=top_products.values, y=top_products.index)
plt.title("Top 10 Most Reviewed Products")
plt.xlabel("Number of Reviews")
plt.ylabel("Product ID")
plt.show()
```



In [48]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='Score', y='HelpfulnessNumerator', data=data)
plt.title("Helpfulness vs Rating")
plt.show()
```



In [54]:
```
# 1️⃣ Import Libraries
import pandas as pd
import re
from sklearn.model_selection import train_test_split
```

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 2️⃣ Load Dataset
data = pd.read_csv("Reviews.csv")  # Make sure Reviews.csv is in your folder
print("Dataset loaded. Number of reviews:", len(data))

# Optional: check columns
print(data.columns)

# ================================
# 3️⃣ Create Sentiment Labels
# Positive (Score >= 4), Negative (Score <= 2), ignore Score = 3
data = data[data['Score'] != 3].copy()
data['Sentiment'] = data['Score'].apply(lambda x: 1 if x >= 4 else 0)  # 1=Positive,
print("Sentiment labels created.")
print(data['Sentiment'].value_counts())

# ================================
# 4️⃣ Text Preprocessing
def clean_text(text):
    text = str(text).lower()  # lowercase
    text = re.sub(r'[^a-z\s]', '', text)  # remove punctuation/numbers
    text = re.sub(r'\s+', ' ', text).strip()  # remove extra spaces
    return text

data['clean_text'] = data['Text'].apply(clean_text)
print("Text preprocessing completed.")

# ================================
# 5️⃣ Train-Test Split
X = data['clean_text']
y = data['Sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Train-test split done. Training size:", len(X_train), "Testing size:", len(X_t

# ================================
# 6️⃣ Text Vectorization (TF-IDF)
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2), stop_words='englis
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
print("TF-IDF vectorization completed.")

# ================================
# 7️⃣ Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)
print("Model training completed.")

# ================================
# 8️⃣ Evaluate Model
y_pred = model.predict(X_test_vec)

print("✅ Model Evaluation Metrics:")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# ================================
# 9️⃣ Predict New Reviews
def predict_sentiment(review):
    review = clean_text(review)
```

```python
    vec = vectorizer.transform([review])
    pred = model.predict(vec)[0]
    return "Positive" if pred == 1 else "Negative"

# Examples
example_reviews = [
    "This product is amazing and works perfectly!",
    "Terrible experience, very disappointed.",
    "I love this, will buy again.",
    "Waste of money, very poor quality."
]

for review in example_reviews:
    print(f"Review: {review}\nPredicted Sentiment: {predict_sentiment(review)}\n")
```

```
Dataset loaded. Number of reviews: 568454
Index(['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator',
       'HelpfulnessDenominator', 'Score', 'Time', 'Summary', 'Text'],
      dtype='object')
Sentiment labels created.
Sentiment
1    443777
0     82037
Name: count, dtype: int64
Text preprocessing completed.
Train-test split done. Training size: 420651 Testing size: 105163
TF-IDF vectorization completed.
Model training completed.
✅ Model Evaluation Metrics:
Accuracy: 0.9305

Classification Report:
               precision    recall  f1-score   support

           0       0.84      0.68      0.75     16407
           1       0.94      0.98      0.96     88756

    accuracy                           0.93    105163
   macro avg       0.89      0.83      0.86    105163
weighted avg       0.93      0.93      0.93    105163

Confusion Matrix:
 [[11214  5193]
 [ 2116 86640]]
Review: This product is amazing and works perfectly!
Predicted Sentiment: Positive

Review: Terrible experience, very disappointed.
Predicted Sentiment: Negative

Review: I love this, will buy again.
Predicted Sentiment: Positive

Review: Waste of money, very poor quality.
Predicted Sentiment: Negative
```

In [ ]: